APP DEVELOPMENT WITH AI ASSISTANCE

Practical session notes using PyCharm / VS Code and a real-world business application

Practical project: MK Agro – Fertilizer Store Customer & Payment Management System

Session focus: Understand the development process, use AI as a coding assistant, and verify the finished application instead of blindly trusting generated code.

Prepared as speaker notes

You can explain the sections in your own words and demonstrate the project where indicated.

1. Important definitions

Term	Meaning
Application (App)	A software program designed to help users perform particular tasks, such as managing customers, recording payments, or generating reports.
App development	The process of planning, designing, building, testing, deploying, and maintaining a software application.
Frontend / User Interface (UI)	The part of an application that users see and interact with: pages, forms, buttons, tables, and menus.
Backend	The logic that works behind the interface. It processes user actions, applies rules, and communicates with the database.
Database	An organized collection of data that can be stored, searched, updated, and managed.
Database schema	The structure of a database: its tables, columns, data types, relationships, and constraints.
CRUD	Four basic data operations: Create, Read, Update, and Delete.
Library	Reusable code written by others that developers can import to add functionality without building everything from scratch.
Framework	A structured platform that provides tools and conventions for building an application. Streamlit is a Python framework for data-oriented web apps.
IDE	Integrated Development Environment: software that brings code editing, running, debugging, and project tools together.
Debugging	Finding, understanding, and fixing problems in a program.
Deployment	Making an application available for users to access, commonly through a hosted server and a web link.
API	Application Programming Interface: a defined way for two software systems to communicate.
Version control	A way to track changes in code over time. Git is a widely used version-control system.

2. What is app development?

App development is not simply writing code. It is a complete process that starts by understanding a problem and ends with a working application that users can rely on.

Example: A fertilizer shop may record customer purchases and payments in notebooks. Finding a customer's previous balance or checking outstanding payments can take time. App development turns this manual process into a more organized digital workflow.

Main stages of app development:

1. **Problem identification:** Understand what is difficult or inefficient in the current process.

2. **Requirement gathering:** Decide who will use the app and what features they need.
3. **Planning and workflow:** Decide how the user will move through the application and what data must be stored.
4. **Design:** Plan screens, forms, tables, navigation, and the database structure.
5. **Development:** Write code and connect the interface, application logic, and database.
6. **Testing:** Check normal use, incorrect inputs, edge cases, and data accuracy.
7. **Deployment:** Publish the app so intended users can access it.
8. **Maintenance:** Fix bugs, improve features, back up data, and respond to user feedback.

> A useful principle to say aloud: **Good development begins with a clear problem, not with a prompt or a block of code.**

3. Flowchart of a typical app development process

Use this flowchart to explain the overall journey. The decision point reminds us that testing may send us back to development for corrections.

Identify the problem
↓
Gather requirements
↓
Plan workflow, interface and database
↓
Develop the application
↓
Test the application
↓
Are the results correct?
Yes → Deploy | No → Debug and improve, then test again
↓
Maintain and improve

4. What is PyCharm?

PyCharm is an Integrated Development Environment (IDE) made by JetBrains for Python development. It provides a place to write, organize, run, and debug Python projects.

Useful features:

- Code editor with syntax highlighting and automatic suggestions.
- Project and file management for keeping code organized.
- Run and debug tools to execute programs and inspect errors.

- Virtual-environment and package-management support.
- Code navigation, refactoring, and inspections that help identify possible issues.
- Git integration for tracking and managing code changes.

Pros of PyCharm

- Designed specifically for Python, so many Python workflows are built in.
- Helpful code completion, navigation, and debugging tools.
- Useful for larger projects with multiple files and modules.
- Provides integrated tools that reduce the need to switch between applications.

Cons / limitations of PyCharm

- Can use more memory and system resources than lightweight editors.
- The interface and number of features may feel complex to beginners.
- Some advanced features are available only in the paid Professional edition; the Community edition has a more limited feature set.
- It is an IDE, not a substitute for understanding programming, databases, or application design.

PyCharm vs VS Code: PyCharm is a Python-focused IDE. Visual Studio Code is a lightweight, general-purpose code editor that can be extended with Python and other extensions. Both can be used to build Python applications.

5. How an AI assistant helps in app development

AI assistants can support developers at different stages. They are useful collaborators, but they do not replace the developer's responsibility for design, decisions, testing, and security.

Understanding requirements: Turn a rough idea into a list of features, user roles, and requirements.

Planning the project: Suggest a folder structure, modules, database tables, and an implementation sequence.

Designing workflows: Help map the steps a user follows when adding a customer, recording a payment, or viewing a report.

Generating code: Draft functions, UI components, SQL queries, validation logic, and documentation.

Explaining code: Explain what a function, parameter, library, or code block does.

Debugging: Interpret error messages and suggest likely causes and fixes.

Testing: Suggest test cases, sample inputs, and edge cases that the developer should verify.

Improving the app: Suggest usability improvements, performance improvements, and possible future features.

A sensible AI-assisted workflow:

1. Explain the real problem and requirements clearly.
2. Ask AI to help plan the workflow, database design, and project structure.
3. Review the proposed design and make decisions yourself.
4. Ask for code in small, understandable modules rather than accepting a huge code dump.
5. Run the code, test it with realistic data, and read the errors.
6. Share the exact error and relevant code with the assistant for debugging.
7. Review the final result, check data handling and access, and test again before deployment.

Important: AI-generated code can contain mistakes, insecure patterns, or assumptions that do not match the business. Never paste passwords, API keys, private customer information, or other confidential data into an AI assistant. Always review and test generated code.

6. My project: MK Agro – Customer & Payment Management System

Project title: MK Agro – Fertilizer Store Customer and Payment Management System

Project type: Python-based business application with a web interface and database.

Live application: https://mk-agro.streamlit.app/

Source code: github.com/arjumand547/MK-Agro

6.1 Problem statement

In a fertilizer store, customer details, purchases, and payments may be managed manually or across separate records. This can make it difficult to find customer information, review payment history, track outstanding balances, and maintain consistent records. The project aims to organize these business records in one digital application.

6.2 Proposed solution

MK Agro provides a centralized application for managing customer information and payment-related records. Instead of relying only on scattered manual entries, the shop can use a structured interface and database to add, view, and update records. The application is intended to make routine record management more organized and accessible.

6.3 Development process

1. **Understand the shop's workflow:** Identify what information is needed about customers and their payments.
2. **Define requirements:** Decide which records users need to add, view, and update.
3. **Plan the database:** Identify the main entities, such as customers and payment records, and decide how they relate. Confirm the actual schema in the project before presenting table names.
4. **Plan the interface:** Organize the screens and forms around the shopkeeper's tasks.
5. **Build the application:** Use Python and Streamlit for the interface and connect the application to its database.
6. **Implement data operations:** Add the required create, read, and update workflows; include delete only where the application supports it and it is appropriate.
7. **Test with sample records:** Check that customer and payment information is saved and displayed correctly.
8. **Deploy:** Publish the application online and provide the user with its web link.

6.4 Libraries and technologies

Tool / library	Purpose
Python	Main programming language used for the application logic.
Streamlit	Python framework used to build the web-based user interface.
SQLite / sqlite3	Database technology for storing structured records. Python's sqlite3 module is part of the standard library; confirm the deployed project configuration.
Pandas (if used)	Useful for working with tabular data, filtering, and data summaries.
Plotly (if used)	Can be used to create interactive charts and visual reports.

Presenter note: The exact dependencies must be checked in the project's requirements.txt or import statements. Do not claim Pandas or Plotly was used unless they are actually present in your version of MK Agro.

6.5 Software to install

- **Python 3:** Install a compatible Python version.
- **PyCharm or Visual Studio Code:** Choose one IDE/editor for writing and running the project.
- **Python packages:** Install the project's dependencies from requirements.txt when available.
- **Git:** Useful for version control and connecting the project to GitHub.
- **Web browser:** For testing the local app and opening the deployed application.
- **Database viewer (optional):** A SQLite database browser can help inspect tables and records.

Typical setup commands (run from the project folder):

```
python -m venv .venv
# Activate the environment for your operating system
python -m pip install -r requirements.txt
streamlit run app.py
```

The final command assumes the main Streamlit file is named app.py. If your entry file has a different name, use that filename instead.

6.6 Testing and verification

Testing means checking whether the application behaves as expected. Verification means confirming that the requirements have actually been met.

Test	What to verify
Add a customer	A valid customer record is saved and appears in the customer list.
View/search records	The correct customer information is displayed and search/filter results are accurate, if supported.
Record a payment	The payment is saved against the correct customer and the displayed totals/balance update correctly, if implemented.
Edit a record	The intended record changes and unrelated records remain unchanged.
Invalid or empty input	The app shows a clear validation message and does not save incorrect data.
Refresh / reopen	Saved records remain available after reopening, subject to the deployed database's persistence configuration.
Different screen sizes	Important controls and information remain usable on the devices the shop will use.

Do not demonstrate with real customer or payment details in a public session. Use sample data or hide private information.

6.7 Common errors and how to solve them

Common issue	Possible cause and response
ModuleNotFoundError	A package is missing or installed in another Python environment. Activate the project's environment and install the required package.
App does not start	Check the terminal traceback, confirm the correct project folder and entry filename, and run the Streamlit command from that folder.
Database/table not found	Check the database path, table creation logic, and whether the app is using the expected database file.

Common issue	Possible cause and response
Data is not saved	Check the insert/update query, transaction handling, commit behavior, database path, and error messages.
Incorrect balance or totals	Check the business rules and calculation logic. Test with small known examples and compare expected versus displayed results.
Deployment works locally but not online	Check dependency versions, entry file, secrets/configuration, file paths, and hosting logs.
Changes disappear after redeployment	Check the hosting platform's storage and persistence rules. A local SQLite file on some cloud hosts may not be persistent or shared. Use an appropriate persistent database and backups for real business data.

7. Suggested live demonstration

1. Open the deployed MK Agro link in a browser.
2. Briefly explain the problem the app is intended to solve.
3. Show the main navigation and explain the purpose of each visible section.
4. Demonstrate one safe workflow, such as adding a sample customer or viewing a sample record, if the live app supports it.
5. Show how a payment-related record is entered or viewed, if available.
6. Explain that the interface is only one part: the application logic processes actions and the database stores records.
7. Finish by explaining how you tested the app and what you would improve next.

Before the session: Open the app and GitHub repository in advance, test the internet connection, and prepare sample data. Confirm that the deployed app is working and that the buttons/screens you plan to demonstrate are available in the current version.

8. Closing points

- App development is a problem-solving process, not just coding.
- A good application needs clear requirements, a sensible workflow, a suitable database, testing, and maintenance.
- PyCharm and VS Code help organize and develop software; each has different strengths.
- AI can speed up planning, coding, explanation, and debugging, but the developer must review and verify its output.
- A real-world project becomes valuable when it solves an actual user's problem reliably.

Suggested closing line: “The main lesson from this project is that AI can help us build faster, but understanding the problem, designing the system, and verifying the result are still the developer's responsibility.”

Links

Live app: https://mk-agro.streamlit.app/
GitHub repository: https://github.com/arjumand547/MK-Agro

Accuracy note: These notes describe the project at a high level based on the information available. Before presenting exact feature names, database tables, libraries, or errors you personally encountered, check the current code and requirements file so the presentation matches the deployed version.